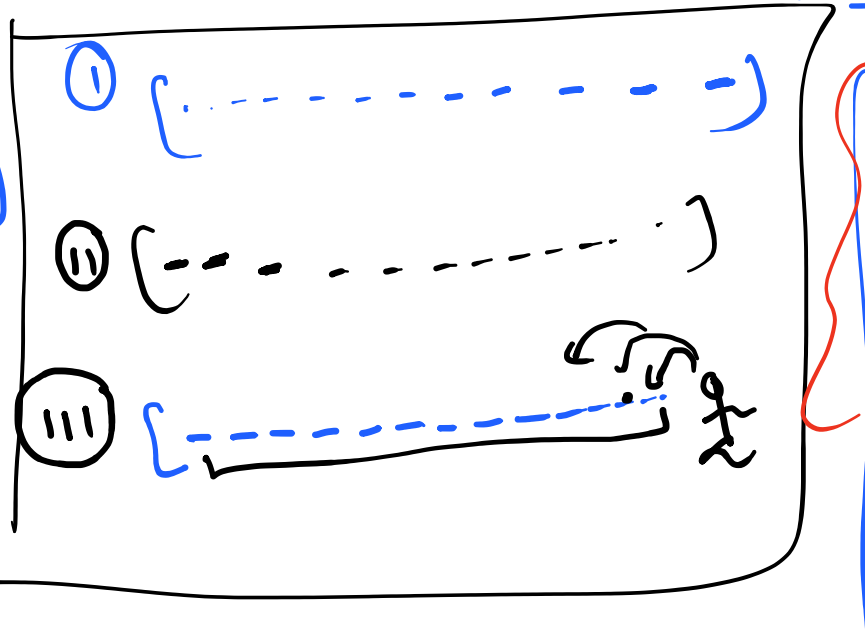
Lec-10

20/05/26 7:08 PM

Sorting :- Inc/dec

[↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓]

⑨ ⑧



① Sorting
- (i) bubble sort
- (ii) selection sort
- (iii) Insertion sort
- (iv) D·N·F
- (v) Merge sort
- (vi) quick sort
- (vii) heap sort
- (viii) counting sort

× bubble sort ×

N→ element
N-1→element

① i=0 4 5 3 2 1
4 5 3 2 1
4 3 5 2 1
4 3 2 5 1
4 3 2 1 ⑤

① ② ③ ④ ⑤

(ii) 4 3 2 1 5
3 4 2 1 5
3 2 4 1 5
3 2 1 4 5

(iii) 3 2 1 4 5
2 3 1 4 5
2 1 3 4 5

(iv) 2 1 3 4 5
1 2 3 4 5

```
for turn=1   turn<n   turn++){
    for(i=0   i<N-Turn   i++){
        if (arr[i] > arr[i+1]){
            swap (arr[i], arr[i+1])
        }
    }
}
```

① 4 5 3 2 1
i=0 0 1 2 3 4
i=1 4 5 3 2 1
i=2 4 3 5 2 1
i=3 4 3 2 5 1
4 3 2 1 ⑤

④

(ii) 4 3 2 1 5
i=0 3 4 2 1 5
③
i=1 3 2 4 1 5
i=2 3 2 1 4 5

(iii) 3 2 1 4 5
i=0 2 3 1 4 5
i=1 2 1 3 4 5
②

(iv) 2 1 3 4 5
i=0 1 2 3 4 5
①

N - turn
5-1=④
5-2=3
5-3=2
5-4=1

idx=2

```
int[] arr = { 4, -1, 5, 3, 2, 1, 7 };
```

0 1 ② 3 4 5 6

arr[mini]

mini= idx→ 2 3 4 5

```
for( i=idx+1  i<N  i++ ){
    if(arr[i] < arr[mini]){
        mini = i
    }
}
```

3 < arr[2]
5
arr[4] < arr[3]
2 3
arr[5] < arr[4]
1 2
arr[6] < arr[5]
7 1

```
public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 4, -1, 5, 3, 2, 1, 7 };
        System.out.println(min_form_ith_index(arr, 2));
    }

    public static int min_form_ith_index(int[] arr, int idx) {
        int mini = idx;
        for (int i = idx; i < arr.length; i++) {
            if (arr[i] < arr[mini]) {
                mini = i;
            }
        }
        return mini;
    }
```

~~idx~~ 2 3 4 5

0 1 2 3 4 5 6
[4, -1, 5 3 2 1 7]

arr[3] < arr[2]
3 5
arr[4] < arr[3]
②< 3

arr[5] < arr[4]
1 2
arr[6] < arr[5]
7 1

① Selection Sort

```
int[] arr = { 4, -1, 5, 3, 2, 1, 7 };
```

0 1 2 3 4 5 6 ⓘ→ ⓘdx

i=0 (0,1)→ -1 4 5 3 2 1 7
0 1 2 3 4 5 6
i=1 (1,5)→ -1 1 5 3 2 4 7
0 1 2 3 4 5 6
i=2 (2,4)→ -1 1 2 3 5 4 7
0 1 2 3 4 5 6
i=3 (3,3)→ -1 1 2 3 5 4 7
i=4 (4,5)→ -1 1 2 3 4 5 7
0 1 2 3 4 5 6
i=5 (5,5)→ -1 1 2 3 4 5 7
i=0

```
int[] arr = { 1, 2, 4, 8, 9, 11, 7 };
```

i=n-1 i=i-1
1 2 4 8 9 7 11
1 2 4 8 7 9 11
1 2 4 7 8 9 11
0 1 2

item= arr[i] ⑦
j = i-1 =

⑨>⑦
8>7
④>⑦

Code

```
while (arr[j] > item){
    arr[j+1] = arr[j];
    arr[j]= item
    j--
}
```

(j+1) return→ correct possition

```
int[] arr = { 9, 8, 1, 7, 4, 2, 11 };
```

0 1 2 3 4 5 6

[9] → sorted

i=1 [9,8] → [8, 9 1 7 4 2 11]
0 1 0 1 2
i=2 [8 9 1] → 1 8 9 7 4 2 11
i=3 1 8 9 ⑦ → 1 7 8 9 4 2 11
i=4 1 7 8 9 4 → [1 4 7 8 9 2 11]
i=

```
public static void InsertLastElement(int[] arr, int i) {
        // TODO Auto-generated method stub
        int item = arr[i];
        int j = i - 1;
        while (arr[j] > item) {
            arr[j + 1] = arr[j];
            arr[j] = item;
            j--;
        }
        // return j+1
    }
```

→ 2
→ 2
j>=0

i=3
0 1 2 3
X [4 5 7 ②]
⑦>2 [4 5 2 7]
5>2 [4 2 5 7]
4>2 ⓪ [2 4 5 7]

Line end

[5 -1 3, -9, 11 -1, 3]
0 1 2 3 4 5 6

5	-1	3	-9	11	-1
5 -1	-1 3	3 -9	-9 11	11 -1	-1 3
5 -1 3	-1 3 -9	3 -9 11	-9 11 -1	11 -1 3	3
5 -1 3 -9	-1 3 -9 11	3 -9 11 -1	-9 11 -1 3		
5 -1 3 -9 11	-1 3 -9 11 -1	3 -9 11 -1 3			
5 -1 3 -9 11 -1	-1 3 -9 11 -1 3				
5 -1 3 -9 11 -1 3					

ans= 8 10 15

[5 3 -9 11 -2 7]
0 1 2 3 4 5

i j
5 5 [0-0]
8 5 3 [0-1]
-1 5 3 -9 [0-2]
10 5 3 -9 11 [0-3]
8 5 3 -9 11 -2 [0-4]
⑮ 5 3 -9 11 -2 7 [0-5]
x

i j
3 3 [1-1]
-6 3 -9 [1-2]
5 3 -9 11 [1-3]
3 3 -9 11 -2 [1-4]
⑩ 3 -9 11 -2 7 [1-5]
x

i j
-9 -9 2-2
2 -9 11 2-3
0 -9 11 -2 2-4
7 -9 11 -2 7 2-5

11 11 3-3
11-2 3-4
9 11 -2 7 3-5
16

-2 -2 4-4
-2 7
5 4-5

7 7 5-5

int ans= -∞

```
for(i=0  i<arr.len  i++){
    int sum=0
    for(j=i  j<arr.len  j++){
        sum = sum+arr[j]
        ans = max (sum, ans)
    }
}
```

[5 3 -12 11 -2 7]
0 1 2 3 4 5

5 ← 5 0-0
8 ← 5 3 0-1
-4 ← 5 3 -12 0-2
7 ← 5 3 -12 11 0-3
5 ← 5 3 -12 11 -2 0-4
⑫ — 5 3 -12 11 -2 7 0-5

```
for(i=0 i<n i++){
    sum+= arr[i]
    ans=max (ans, sum)
}
```

-4
0 1 2 3 4 5 6 7 8 9
[5 3 -12 11 -2 7 -19, 2 -1, 3]
x
ans = -∞ 5 8 16 ~~15~~

```
public static int Maximum_Sum(int[] arr) {
        int ans = Integer.MIN_VALUE;
        int sum = 0;
        for (int i = 0; i < arr.length; i++) {
            sum = sum + arr[i];
            ans = Math.max(ans, sum);
        }
        return ans;
    }
```

sum=0+5
=5+3
=8-12
=-4 <0

if (sum<0){
sum=0
}

sum=0+11
=11-2
= 9+7
=16-19
=-3

sum=0+2
=2-1
=1+3
=④

2 -3 4 7 -9 6

```
public static int Maximum_Sum(int[] arr) {
        int ans = Integer.MIN_VALUE;
        int sum = 0;
        for (int i = 0; i < arr.length; i++) {
            sum = sum + arr[i];
            ans = Math.max(ans, sum);
            if(sum<0) {
                sum=0;
            }
        }
        return ans;
    }
```

-2 -3

0 1 2
[-4 -2 -3]
ans = -∞ -4 -2
sum=0-4=-4
sum=0-2=-2
sum=0-3=-3
sum=0

7 ④
8→⑧

25/2=12

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

(n/2+1)

25
⑬ 12

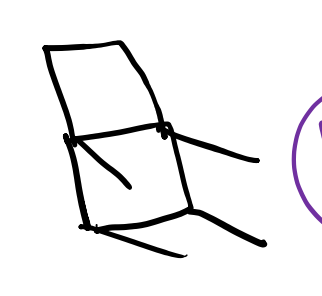
(+) (-)

0 1 2 3 4 5 6 7 8
[2 7 3 7 7 5 7 3 7]

8/2+1= ⑤

e = 2 7 3 7 3 7
vote= 1 0 1 0 1 0 1 ②

(N/2+1)

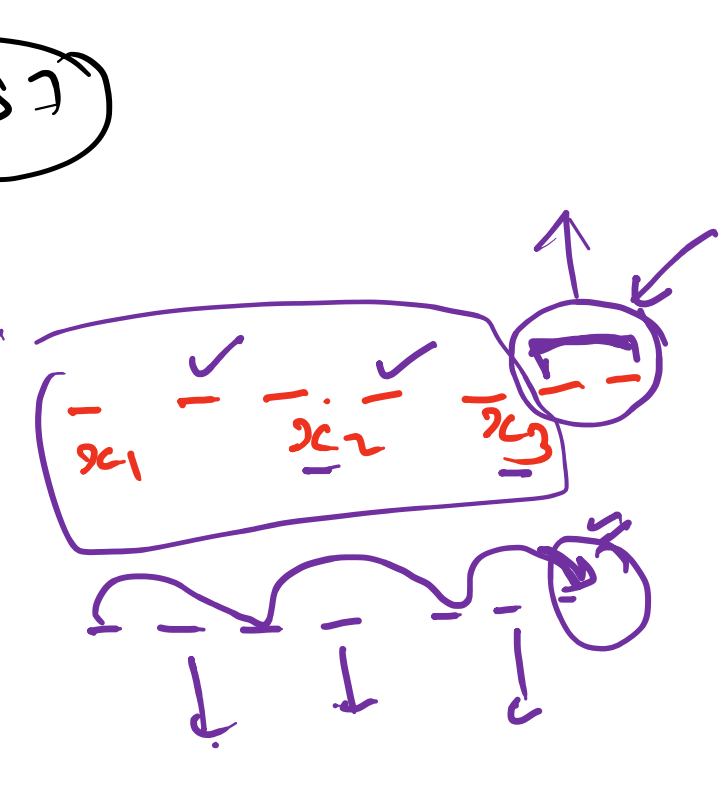


(last)↑

(2 3 5 7)

e=2 7 3 7 3 7
v=1 0 1 0 1 0 1 ① ② ①

2 7 3 7 3 7 7 5 7

```
for(i=1   i<n   i++){
    if(arr[i]==e){
        vote++
    }
    else{
        vote--
        if(vote==0){
            e=arr[i]
            vote=1
        }
    }
}
```